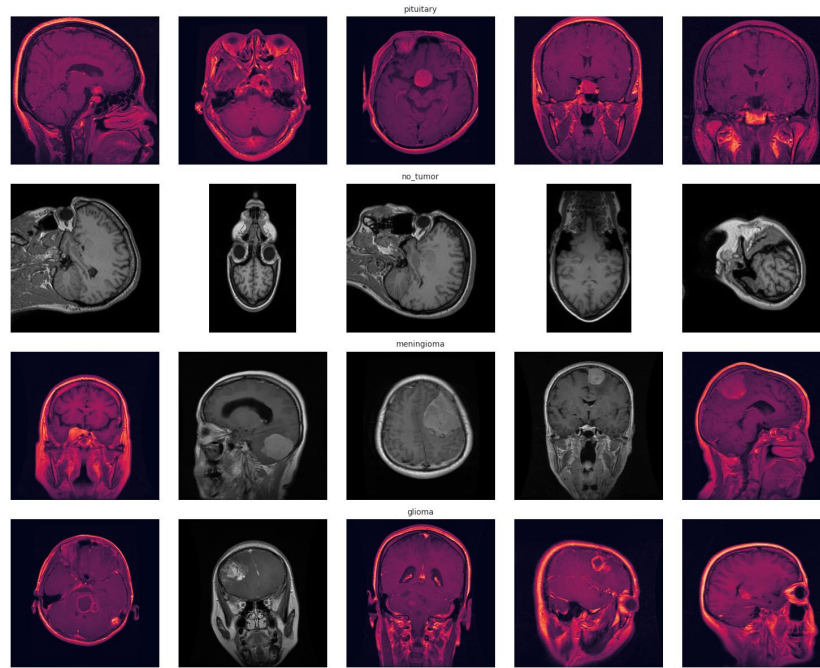


Brain Tumor Classification using Layer-wise Linear + Softmax Hybrid Attention



```
import numpy as np
import pandas as pd
import os

root_path = '/kaggle/input/brisc2025/brisc2025/classification_task/train'

image_paths = []
labels = []

for label in os.listdir(root_path):
    label_path = os.path.join(root_path, label)
    if os.path.isdir(label_path):
        for img_file in os.listdir(label_path):
            if img_file.lower().endswith(('.png', '.jpg', '.jpeg')):
                image_paths.append(os.path.join(label_path, img_file))
                labels.append(label)

df = pd.DataFrame({'image_path': image_paths, 'label': labels})

df
```

	image_path	label
0	/kaggle/input/brisc2025/brisc2025/classificati...	pituitary
1	/kaggle/input/brisc2025/brisc2025/classificati...	pituitary
2	/kaggle/input/brisc2025/brisc2025/classificati...	pituitary
3	/kaggle/input/brisc2025/brisc2025/classificati...	pituitary
4	/kaggle/input/brisc2025/brisc2025/classificati...	pituitary
...	...	...
4995	/kaggle/input/brisc2025/brisc2025/classificati...	glioma

```
4996 /kaggle/input/brisc2025/brisc2025/classificati... glioma
4997 /kaggle/input/brisc2025/brisc2025/classificati... glioma
4998 /kaggle/input/brisc2025/brisc2025/classificati... glioma
4999 /kaggle/input/brisc2025/brisc2025/classificati... glioma
```

```
[5000 rows x 2 columns]
```

```
df.shape
```

```
(5000, 2)
```

```
df.columns
```

```
Index(['image_path', 'label'], dtype='object')
```

```
df.duplicated().sum()
```

```
0
```

```
df.isnull().sum()
```

```
image_path    0
```

```
label         0
```

```
dtype: int64
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 5000 entries, 0 to 4999
```

```
Data columns (total 2 columns):
```

#	Column	Non-Null Count	Dtype
0	image_path	5000 non-null	object
1	label	5000 non-null	object

```
dtypes: object(2)
```

```
memory usage: 78.3+ KB
```

```
df['label'].unique()
```

```
array(['pituitary', 'no_tumor', 'meningioma', 'glioma'], dtype=object)
```

```
df['label'].value_counts()
```

```
label
```

```
pituitary    1457
```

```
meningioma   1329
```

```
glioma       1147
```

```
no_tumor     1067
```

```
Name: count, dtype: int64
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
sns.set_style("whitegrid")
```

```
fig, ax = plt.subplots(figsize=(8, 6))
```

```

sns.countplot(data=df, x="label", palette="viridis", ax=ax)

ax.set_title("Distribution of Tumor Types", fontsize=14, fontweight='bold')
ax.set_xlabel("Tumor Type", fontsize=12)
ax.set_ylabel("Count", fontsize=12)

for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='bottom', fontsize=11, color='black',
                xytext=(0, 5), textcoords='offset points')

plt.show()

label_counts = df["label"].value_counts()

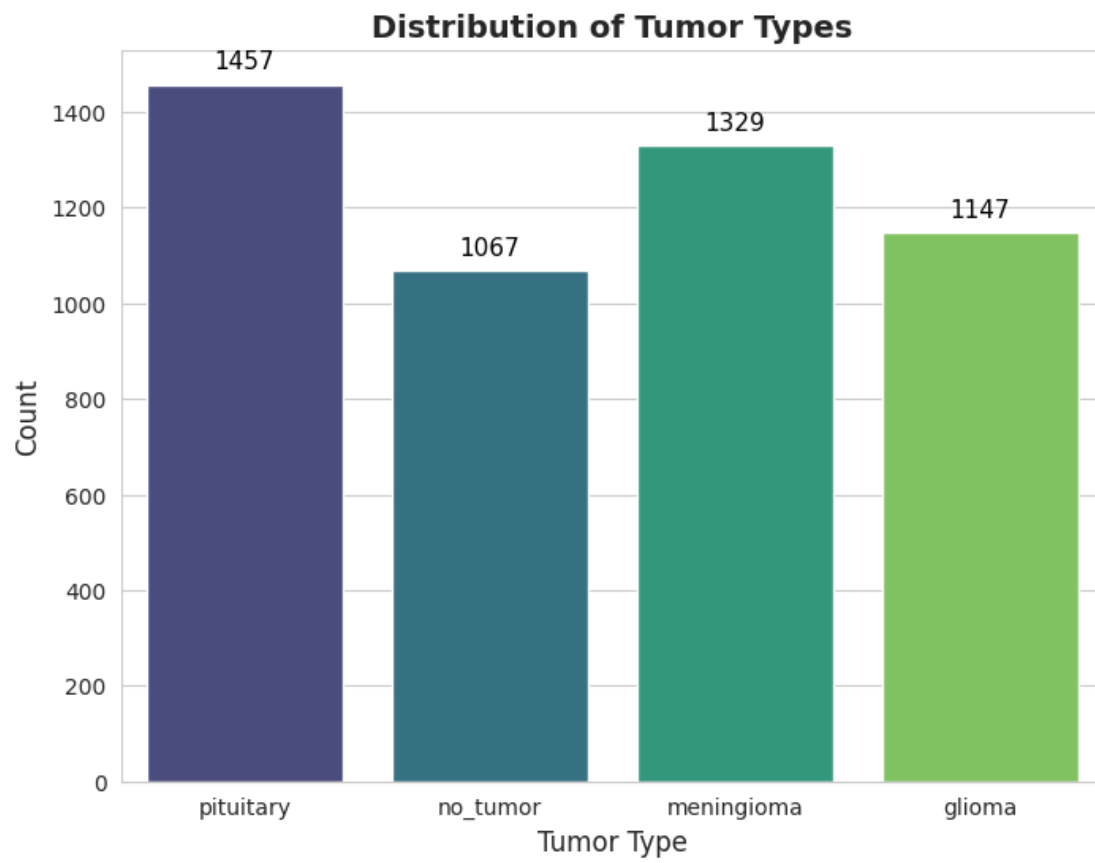
fig, ax = plt.subplots(figsize=(10, 8))
colors = sns.color_palette("viridis", len(label_counts))

ax.pie(label_counts, labels=label_counts.index, autopct='%1.1f%%',
        startangle=140, colors=colors, textprops={'fontsize': 8, 'weight':
        'bold'},
        wedgeprops={'edgecolor': 'black', 'linewidth': 1})

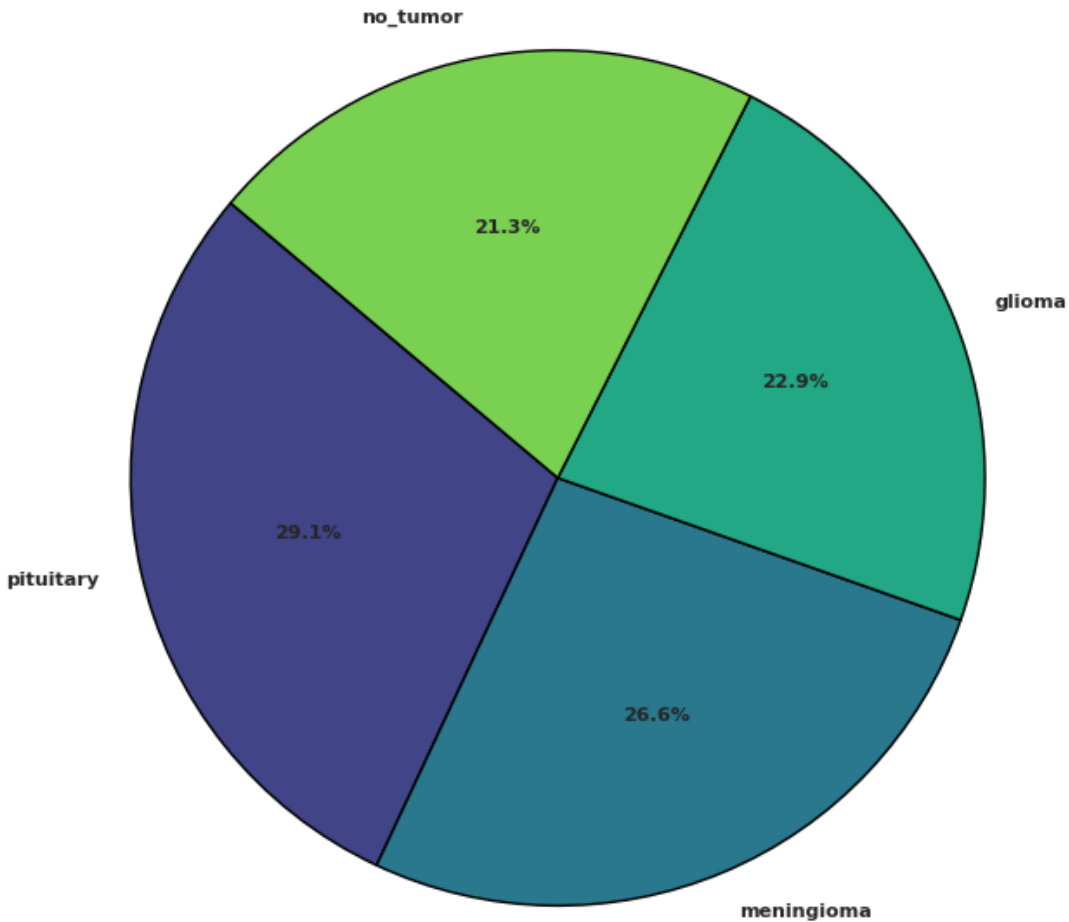
ax.set_title("Distribution of Tumor Types - Pie Chart", fontsize=14,
fontweight='bold')

plt.show()

```



Distribution of Tumor Types - Pie Chart



```
from PIL import Image

num_images = 5

unique_labels = df['label'].unique()

plt.figure(figsize=(15, len(unique_labels) * 3))

for row_idx, label in enumerate(unique_labels):

    label_images = df[df['label'] ==
label].head(num_images)['image_path'].tolist()

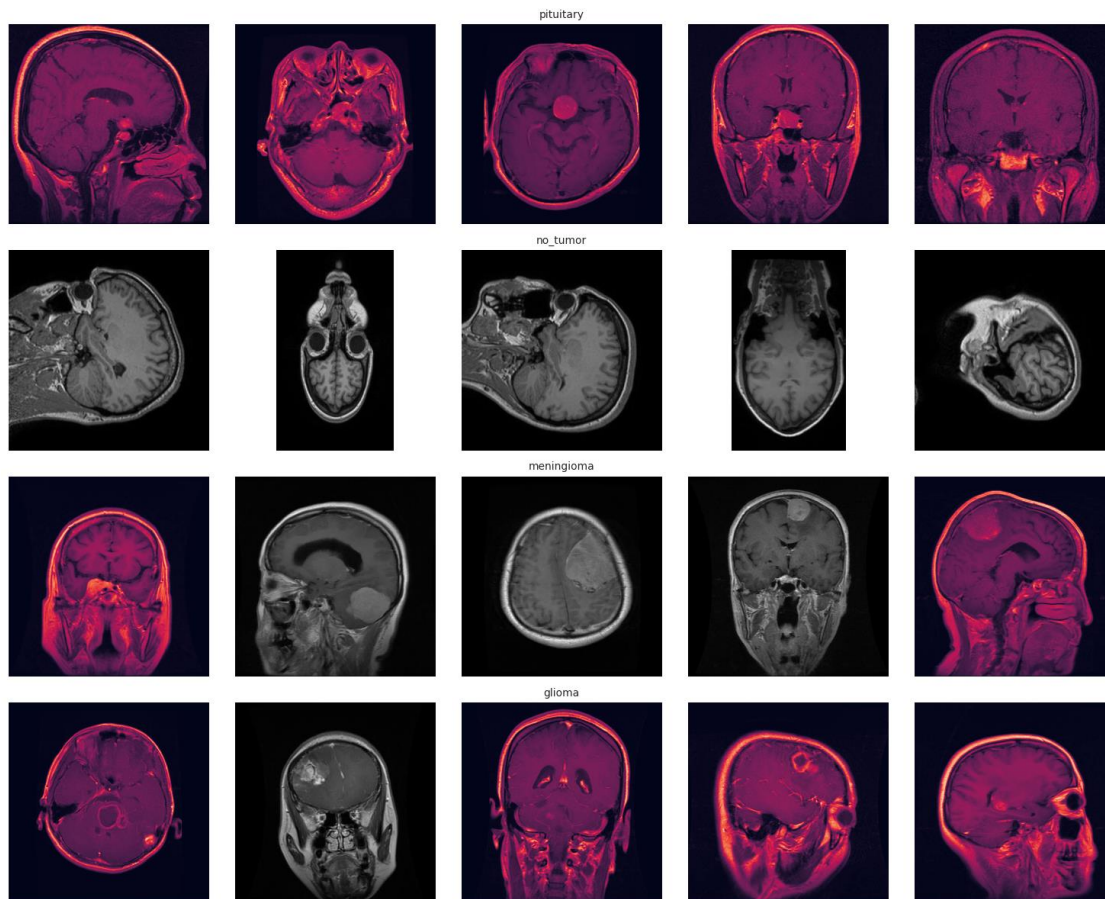
    for col_idx, img_path in enumerate(label_images):
        plt_idx = row_idx * num_images + col_idx + 1
        plt.subplot(len(unique_labels), num_images, plt_idx)
        img = Image.open(img_path)
        plt.imshow(img)
```

```

plt.axis('off')
if col_idx == 2:
    plt.title(label, fontsize=10)

plt.tight_layout()
plt.show()

```



```

import warnings
warnings.filterwarnings('ignore')

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms, models
from PIL import Image
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split

train_df, val_df = train_test_split(df, test_size=0.2, stratify=df['label'],
random_state=42)

max_samples = train_df['label'].value_counts().max()

```

```
balanced_train_df = train_df.groupby('label', group_keys=False).apply(
    lambda x: x.sample(n=max_samples, replace=True, random_state=42)
).reset_index(drop=True)
```

```
balanced_train_df = balanced_train_df[['image_path', 'label']]
```

```
!pip install einops
```

```
Requirement already satisfied: einops in /usr/local/lib/python3.11/dist-packages (0.8.1)
```

```
from einops import rearrange
import sklearn.metrics as metrics
from sklearn.metrics import classification_report, confusion_matrix
```

```
class LinearAttention(nn.Module):
    def __init__(self, dim, heads=4):
        super().__init__()
        self.heads = heads
        self.scale = dim ** -0.5
        self.to_qkv = nn.Linear(dim, dim * 3, bias=False)
        self.to_out = nn.Linear(dim, dim)

    def forward(self, x):
        b, n, d = x.shape
        h = self.heads
        qkv = self.to_qkv(x).chunk(3, dim=-1)
        q, k, v = map(lambda t: rearrange(t, 'b n (h d) -> b h n d', h=h),
            qkv)
        k = k.softmax(dim=-1)
        context = torch.einsum('b h n d, b h n e -> b h d e', k, v)
        out = torch.einsum('b h d e, b h n d -> b h n e', context, q)
        out = rearrange(out, 'b h n d -> b n (h d)')
        return self.to_out(out)
```

```
class SoLABlock(nn.Module):
    def __init__(self, dim, heads=4, use_softmax=False):
        super().__init__()
        self.use_softmax = use_softmax
        self.norm1 = nn.LayerNorm(dim)
        if use_softmax:
            self.attn = nn.MultiheadAttention(dim, heads)
        else:
            self.attn = LinearAttention(dim, heads)
        self.norm2 = nn.LayerNorm(dim)
        self.ffn = nn.Sequential(
            nn.Linear(dim, dim * 4),
            nn.GELU(),
            nn.Linear(dim * 4, dim)
        )

    def forward(self, x):
        residual = x
        attn_input = self.norm1(x)
        if self.use_softmax:
```

```

        attn_input = attn_input.permute(1, 0, 2)
        out, _ = self.attn(attn_input, attn_input, attn_input)
        out = out.permute(1, 0, 2)
    else:
        out = self.attn(attn_input)
    x = residual + out
    residual = x
    x = residual + self.ffn(self.norm2(x))
    return x

class SoLAVision(nn.Module):
    def __init__(self, num_classes=4, dim=256, depth=6, heads=4):
        super().__init__()
        self.dim = dim
        self.patch_embed = nn.Conv2d(3, dim, kernel_size=16, stride=16)
        num_patches = (224 // 16) ** 2
        self.pos_embed = nn.Parameter(torch.randn(1, num_patches + 1, dim))
        self.cls_token = nn.Parameter(torch.randn(1, 1, dim))
        pattern = ['L', 'L', 'L', 'S', 'L', 'L']
        self.blocks = nn.ModuleList([
            SoLABlock(dim, heads, use_softmax=(p == 'S')) for p in pattern
        ])
        self.norm = nn.LayerNorm(dim)
        self.head = nn.Linear(dim, num_classes)
    def forward(self, x):
        b, c, h, w = x.shape
        x = self.patch_embed(x)
        x = x.flatten(2).transpose(1, 2)
        cls_tokens = self.cls_token.expand(b, -1, -1)
        x = torch.cat((cls_tokens, x), dim=1)
        x = x + self.pos_embed
        for block in self.blocks:
            x = block(x)
        x = self.norm(x[:, 0])
        return self.head(x)

class BrainTumorDataset(Dataset):
    def __init__(self, df, transform=None):
        self.df = df
        self.transform = transform
        self.label_map = {'pituitary': 0, 'no_tumor': 1, 'meningioma': 2,
'glioma': 3}
        self.classes = list(self.label_map.keys())
    def __len__(self):
        return len(self.df)
    def __getitem__(self, idx):
        img_path = self.df['image_path'].iloc[idx]
        label_str = self.df['label'].iloc[idx]
        label = self.label_map[label_str]
        image = Image.open(img_path).convert('RGB')
        if self.transform:
            image = self.transform(image)

```



```

        return image, label

transform_train = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])
])
transform_val = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])
])

train_dataset = BrainTumorDataset(balanced_train_df,
transform=transform_train)
val_dataset = BrainTumorDataset(val_df, transform=transform_val)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True,
num_workers=2)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False,
num_workers=2)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

model = SoLAVision()
model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
train_losses = []
train_accs = []
val_losses = []
val_accs = []

num_epochs = 20

for epoch in range(1, num_epochs + 1):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0
    for data in train_loader:
        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()

```

```

        optimizer.step()
        running_loss += loss.item()
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
    train_loss = running_loss / len(train_loader)
    train_acc = 100 * correct / total
    train_losses.append(train_loss)
    train_accs.append(train_acc)
    print(f'Epoch [{epoch}/{num_epochs}] Train Loss: {train_loss:.4f}, Acc:
{train_acc:.2f}%')
    model.eval()
    running_loss = 0.0
    correct = 0
    total = 0
    with torch.no_grad():
        for data in val_loader:
            inputs, labels = data
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            running_loss += loss.item()
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    val_loss = running_loss / len(val_loader)
    val_acc = 100 * correct / total
    val_losses.append(val_loss)
    val_accs.append(val_acc)
    print(f'Epoch [{epoch}/{num_epochs}] Val Loss: {val_loss:.4f}, Acc:
{val_acc:.2f}%')

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(range(1, num_epochs+1), train_losses, label='Train Loss')
plt.plot(range(1, num_epochs+1), val_losses, label='Val Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.title('Loss Curve')
plt.subplot(1, 2, 2)
plt.plot(range(1, num_epochs+1), train_accs, label='Train Acc')
plt.plot(range(1, num_epochs+1), val_accs, label='Val Acc')
plt.xlabel('Epochs')
plt.ylabel('Accuracy (%)')
plt.legend()
plt.title('Accuracy Curve')
plt.show()

model.eval()

y_true = []

```

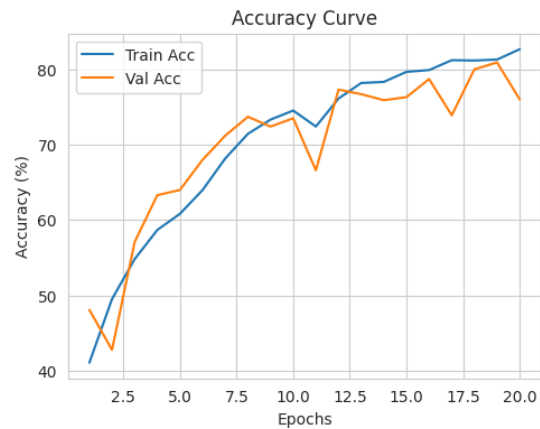
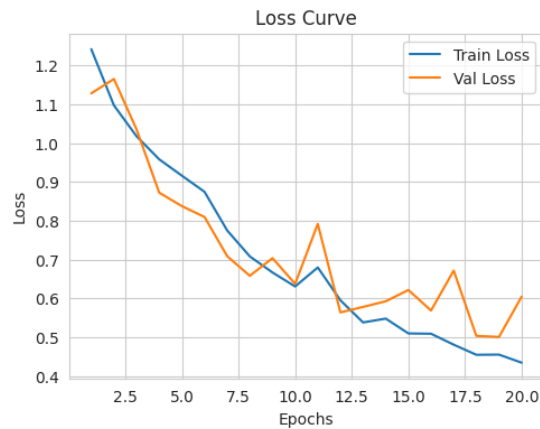
```
y_pred = []
```

```
with torch.no_grad():
    for data in val_loader:
        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = model(inputs)
        _, predicted = torch.max(outputs, 1)
        y_true.extend(labels.cpu().numpy())
        y_pred.extend(predicted.cpu().numpy())

print(classification_report(y_true, y_pred,
target_names=val_dataset.classes))
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=val_dataset.classes, yticklabels=val_dataset.classes)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()
```

```
Epoch [1/20] Train Loss: 1.2422, Acc: 41.09%
Epoch [1/20] Val Loss: 1.1284, Acc: 48.10%
Epoch [2/20] Train Loss: 1.0973, Acc: 49.53%
Epoch [2/20] Val Loss: 1.1654, Acc: 42.80%
Epoch [3/20] Train Loss: 1.0176, Acc: 54.83%
Epoch [3/20] Val Loss: 1.0352, Acc: 57.10%
Epoch [4/20] Train Loss: 0.9582, Acc: 58.69%
Epoch [4/20] Val Loss: 0.8723, Acc: 63.30%
Epoch [5/20] Train Loss: 0.9162, Acc: 60.84%
Epoch [5/20] Val Loss: 0.8376, Acc: 64.00%
Epoch [6/20] Train Loss: 0.8745, Acc: 63.99%
Epoch [6/20] Val Loss: 0.8096, Acc: 68.00%
Epoch [7/20] Train Loss: 0.7749, Acc: 68.18%
Epoch [7/20] Val Loss: 0.7086, Acc: 71.20%
Epoch [8/20] Train Loss: 0.7080, Acc: 71.44%
Epoch [8/20] Val Loss: 0.6580, Acc: 73.70%
Epoch [9/20] Train Loss: 0.6666, Acc: 73.33%
Epoch [9/20] Val Loss: 0.7035, Acc: 72.40%
Epoch [10/20] Train Loss: 0.6305, Acc: 74.53%
Epoch [10/20] Val Loss: 0.6373, Acc: 73.50%
Epoch [11/20] Train Loss: 0.6795, Acc: 72.40%
Epoch [11/20] Val Loss: 0.7918, Acc: 66.60%
Epoch [12/20] Train Loss: 0.5948, Acc: 76.12%
Epoch [12/20] Val Loss: 0.5636, Acc: 77.30%
Epoch [13/20] Train Loss: 0.5377, Acc: 78.18%
Epoch [13/20] Val Loss: 0.5778, Acc: 76.70%
Epoch [14/20] Train Loss: 0.5477, Acc: 78.33%
Epoch [14/20] Val Loss: 0.5924, Acc: 75.90%
Epoch [15/20] Train Loss: 0.5094, Acc: 79.66%
Epoch [15/20] Val Loss: 0.6214, Acc: 76.30%
Epoch [16/20] Train Loss: 0.5086, Acc: 79.89%
```

Epoch [16/20] Val Loss: 0.5684, Acc: 78.70%
Epoch [17/20] Train Loss: 0.4804, Acc: 81.20%
Epoch [17/20] Val Loss: 0.6714, Acc: 73.90%
Epoch [18/20] Train Loss: 0.4544, Acc: 81.16%
Epoch [18/20] Val Loss: 0.5031, Acc: 80.00%
Epoch [19/20] Train Loss: 0.4548, Acc: 81.29%
Epoch [19/20] Val Loss: 0.5003, Acc: 80.90%
Epoch [20/20] Train Loss: 0.4340, Acc: 82.66%
Epoch [20/20] Val Loss: 0.6043, Acc: 76.00%



	precision	recall	f1-score	support
pituitary	0.86	0.92	0.89	292
no_tumor	0.96	0.75	0.84	213
meningioma	0.56	0.87	0.68	266
glioma	0.94	0.44	0.60	229
accuracy			0.76	1000
macro avg	0.83	0.74	0.75	1000
weighted avg	0.82	0.76	0.76	1000

